

Code Translation using Large Language Models: What's the role of the Research Software Engineer?

Stephen Thompson, Asif Tamuri

Code Translation using Large Language Models: What's the role of the Research Software Engineer?



Stephen Thompson, Asif Tamuri

UCL Centre for Advanced Research Computing and UCL Innovative Clinical Trials Unit

2026-09-09

Today's Talk

- Brief Introduction (UCL ICTU, UCL ARC)
- Code Translation Motivation
- Code Translation Mechanics
- Workshop Results

Innovative Clinical Trials Unit and ARC

- Experts in clinical trial design and analysis - statisticians.
 - Impact through successful clinical trials / publications.
 - A lot of software written in Stata.
- Research software engineers
 - Collaboration to create impact through software.

About Stata

- Closed source commercial statistics software.
- Supports community contribution with a package library and publication through the Stata journal.
- Stata journal encourages good software practice - tests, documentation.

Motivation: Example artbin.

Motivation: Example artbin.

- High quality software (including documentation and tests)
- Demonstrated impact - used and cited in published trials.
- Open source (GPL3) - free to download/modify
 - need Stata License to run.
- Stata community is tiny compared to R or Python.
- Can we increase impact by translation.

Stata2R: Practicalities

- Pre-requisites:
 - Well written/documented/tested Stata code 🙌
 - Language Expertise (Stata and R)
 - Domain Expertise (Statistics)
 - Time (funding)
- Lets leverage advances in agent based code translation?

Agentic Coding (Claude Code)

- Users communicate in plain text (for example “translate artbin to R”)
 - Non deterministic (stochastic sampling from many possible answers).
 - Will usually do what you ask, which may not be a good thing.
- We developed the **Stata2R** “plugin” with 4 “skills” to limit the impact of non deterministic behaviour and unquestioning behaviour.
- We delivered a 4 hour workshop to use the plugin.

What’s in a Plugin / Skill

Skills are defined in markdown.

- Plugins are collections of skills that are easy to install
- ```
/plugins marketplace add https://github.com/stata-translations/Stata2R.git
/plugin install stata-translation
/reload-plugins
/stata-translation:hello
```

```
/stata-translation:translation-strategy
```

```
/stata-translation:stata-to-pseudocode
```

```
/stata-translation:pseudocode-to-r
```

```
/stata-translation:add-missing-tests
```

## Plugin Skill 1: Translation Strategy

- Summarise the purpose of the library.
- Is it worth translating? (existing R libraries, users etc)
- Is the documentation and testing sufficient to enable a good translation.

## Plugin Skill 2: Stata to Pseudocode

- Evidence that two stage translation creates better code.
- Enables a human review step.
- Use any documentation files (.sthlp) to inform the translation.
- Ignore all files in the testing directory.

## Plugin Skill 3: Pseudocode to R

- Use the pseudocode to create an R package.
- Create automated tests.
- Port Stata documentation and examples to R (create vignettes)
- Port any dialogues (graphical user interfaces) to R.
- Copy over licence file and create README, CONTRIBUTING, files etc.

## Plugin Skill 4: Add Missing Stata Tests to R

- Port the Stata tests to R and confirm they pass.
- Document Stata to R test correspondence and verify results.

## Developed a training program.

We developed and delivered a [Workshop](#) to 10 statisticians aiming to:

- Teach statisticians to use Claude to translate code.
- Use statisticians to review the quality of resulting translations.
  - [Pre Workshop Surveys](#)
  - [Post work shop survey](#)

## Pre-Workshop: Stata and R Fluency

## Stata fluency vs. R fluency, by role

Each dot is one respondent. Since fluency has only 3 levels (No / Occasional / Fluent), dots are spread out within each cell so overlapping respondents stay visible — position within a cell carries no meaning beyond the cell itself.

R fluency

Fluent

Occasional

No

No

Occasional

Fluent

Stata fluency

Statistician (6)

Statistician & Researcher (3)

Researcher (2)

Research Software Engineer (1)

Researcher & Data Scientist (1)

The dominant pattern (8 of 13 respondents) is Stata-fluent / R-occasional, spanning every role. The 3 respondents who are R-fluent instead of Stata-fluent are the ones with a second role attached (Statistician & Researcher, Researcher & Data Scientist) plus one solo Statistician — consistent with the negative Stata↔R correlation shown separately ( $\rho = -0.59$ ).

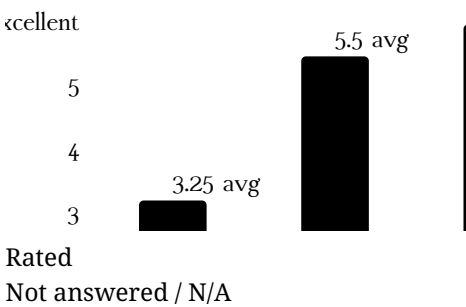
View as table

| Role                        | Stata      | R          | Respondents |
|-----------------------------|------------|------------|-------------|
| Statistician                | Occasional | Occasional | 1           |
| Statistician                | Fluent     | Occasional | 4           |
| Statistician                | Occasional | Fluent     | 1           |
| Researcher                  | Fluent     | Occasional | 2           |
| Statistician & Researcher   | Fluent     | Occasional | 2           |
| Statistician & Researcher   | Occasional | Fluent     | 1           |
| Researcher & Data Scientist | Occasional | Fluent     | 1           |
| Research Software Engineer  | No         | Occasional | 1           |

**Post-Workshop: Skills assessment (Target language quality).**

## Model vs. target language code generation

9 respondents. One bar per model, height = mean rating on the survey's 1–6 scale (6 = Excellent).



Sonnet rates highest here (5.5 avg, from a 5 and a 6 "Excellent"). Opus averages 3.25 across all four of its attempts (4, 3, 2, 4). Haiku and Haiku 4.5 both scored a 2 on their single attempt each — the lowest ratings in the set. Fable's respondent marked this N/A, consistent with that run stopping early (out of credits) before reaching target-language code generation.

With 1–4 responses per model, this isn't enough to rank models on code generation quality — it's a snapshot of these 9 attempts.

View as table

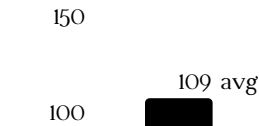
| Model     | Mean (1–6) | Individual responses |
|-----------|------------|----------------------|
| Opus      | 3.25       | 4, 3, 2, 4           |
| Sonnet    | 5.5        | 5, 6 (Excellent)     |
| Fable     | —          | N/A                  |
| Haiku 4.5 | 2          | 2                    |
| Haiku     | 2          | 2                    |

## Post-Workshop: Skills assessment Example (Time Taken).

# Model used vs. time taken to translate

9 respondents. One bar per model, height = mean time.

30 min



Mean time (minutes)

Single, incomplete run (Fable, ~30 min – ran out of credits)

Opus's average (109 min) is pulled up by a single 180-minute attempt — its four attempts actually ranged 60–180 (see the table below). Haiku and Haiku 4.5 were the fastest (5–10 min), but each is a single data point. Fable's 30 minutes isn't a completed translation — that run stopped when the respondent ran out of tokens/credits, so it understates how long Fable would actually take.

With 1–4 attempts per model and effort/complexity varying by task, this is far too little data to conclude one model is reliably faster than another — read it as what happened in these 9 attempts, not a benchmark.

View as table

| Model     | Mean (min) | Range  | n |
|-----------|------------|--------|---|
| Opus      | 109        | 60–180 | 4 |
| Sonnet    | 75         | 60–90  | 2 |
| Fable     | 30*        | —      | 1 |
| Haiku 4.5 | 5          | —      | 1 |
| Haiku     | 10         | —      | 1 |

| Model     | Time (min) | Note                         |
|-----------|------------|------------------------------|
| Fable     | 30         | stopped – ran out of credits |
| Opus      | 60         |                              |
| Opus      | 75         | "with lot of debugging"      |
| Opus      | 120        |                              |
| Opus      | 180        |                              |
| Sonnet    | 60         |                              |
| Sonnet    | 90         |                              |
| Haiku 4.5 | 5          |                              |
| Haiku     | 10         |                              |

## Post-Workshop: Concerns Work Cloud

Post-Workshop Survey: Concerns

Stata Translation Workshop · Post-Workshop Survey

### Concerns about using LLMs for code translation

Drawn only from "What concerns (if any) do you have with using large language models for code translation?" (all 9 respondents answered). Same style as the earlier clouds: packed into a circle, right angles only, tight regular spacing. Size and color track how many respondents raised each concern; generic subject-matter words (code, Stata, Claude...) stay small and gray.

Package

**Bigger & more colorful** = raised by more respondents · **small, gray & faint** = general topic, not a concern in itself

**Two concerns tie for the top spot** (2 of 9 respondents each): fearing the translated code is subtly wrong and hard to catch, and needing better ways to test/control the model's output. Every other concern was raised by exactly one respondent — overconfidence in incorrect code, silently simplified code going unflagged, checking accuracy on large packages, code that isn't idiomatic to the target language, maintaining code nobody wrote by hand, model stochasticity, who's accountable for the output, trusting it for real research decisions, unfamiliar shell/PATH setup, and translating new features incrementally rather than redoing everything.

View concerns as table, with the original text

| Concern                  | Respondents | Original text                                                                                                            |
|--------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------|
| Fear of hidden errors    | 2           | "the code may be wrong and it would take a lot of time to find the errors" · "worry that translated code might be wrong" |
| Testing & output control | 2           | "a way to control what the model spits out, by creating very good testing files" · "how to test the translated code"     |
| Overconfidence / false   | 1           | "Code that appears to be correct but is not - over confidence."                                                          |

|                                 |   |                                                                                                |
|---------------------------------|---|------------------------------------------------------------------------------------------------|
| correctness                     |   |                                                                                                |
| Silent simplification           | 1 | "My code got simplified by Claude and it just put a comment... # simplified"                   |
| Flagging changes                | 1 | "a skill saying to Claude to report back if it does any simplifications or approximations"     |
| Manual verification effort      | 1 | "check it line-by-line and do extensive testing"                                               |
| Accuracy at scale               | 1 | "How do we check accuracy, especially for large complex packages"                              |
| Maintainability                 | 1 | "How can packages be updated/maintained/checked if not human-written?"                         |
| Code elegance & idiomatcity     | 1 | "Lack of elegance (e.g. using optimal structures/organisation/logic for each coding language)" |
| Stochasticity & reproducibility | 1 | "The intrinsic stochasticity..."                                                               |
| Accountability                  | 1 | "the question of who is responsible for the code"                                              |
| High-stakes trust               | 1 | "particularly if you're trusting it for scientific research and decision making"               |
| Shell & script unfamiliarity    | 1 | "Lots of requests are shell script code which I didn't quite understand"                       |
| Environment setup               | 1 | "Need to make sure to set up PATH variables correctly"                                         |
| Incremental re-translation      | 1 | "find a way to translate... new feature... without re-translating everything"                  |

## Post-Workshop: Ideas for Improvements / Future Work.

Post-Workshop Survey: What People Want Improved

Stata Translation Workshop · Post-Workshop Survey

## What people want improved

Drawn from the "concerns," "RSE role," and "other comments" free-text answers (9 respondents). Words are packed edge-to-edge into a circular cloud, upright or rotated a clean 90° only — no diagonal tilts. Concrete

improvement asks are large and colorful; generic subject-matter words (testing, code, Stata...) stay small and gray — they're topic, not an ask.

Environment & P/

**Bigger & more colorful** = a stronger, more specific improvement point ·  
**small, gray & faint** = general topic, not an ask

**Biggest ask:** a reliable way to verify translated code is actually correct — 5 of 9 respondents raised this independently (code that "looks right but isn't," wanting line-by-line checks, no clear way to check accuracy on large packages, an unverifiable self-correction claim from Claude). Everything else was raised by 1–2 respondents each; treat the smaller phrases as individual ideas worth watching, not consensus. Color here is decorative (one hue per phrase, cycled) rather than meaningful — it's there to make each phrase easy to pick out, not to group them.

View themes as table, with a representative quote

| Theme                           | Respondents | Representative quote                                                                                                           |
|---------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| Verifying correctness           | 5           | "Code that appears to be correct but is not – over confidence."                                                                |
| Flagging silent changes         | 2           | "My code got simplified by Claude and it just put a comment saying # simplified... report back if it does any simplifications" |
| Understanding model limitations | 2           | "Understanding the limitations, and... where it may need support."                                                             |
| Line-by-line review             | 1           | "I would want to check it line-by-line and do extensive testing."                                                              |
| Human-readable test scripts     | 1           | "the test script ... wasn't very user-friendly (used testthat package)"                                                        |
| Reporting discrepancies         | 1           | "discrepancies between my STATA documentation and STATA code... did not flag this issue"                                       |
| Verifying self-corrections      | 1           | "Claude claimed to find an error in its translation and correct it; I was unable to verify this"                               |
| Idiomatic code structure        | 1           | "Lack of elegance (e.g. using optimal structures/organisation/logic for each coding language)"                                 |
| Standardized                    |             |                                                                                                                                |

|                                |   |                                                                                                          |
|--------------------------------|---|----------------------------------------------------------------------------------------------------------|
| translation frameworks         | 1 | "Designing systems/frameworks to standardise code translation processes."                                |
| Increasing rigour              | 1 | "Increasing the rigour of what we do"                                                                    |
| Drop-in support clinics        | 1 | "Drop-in clinic (e.g. every month) where we could ask for advice"                                        |
| Skill & plugin guidance        | 1 | "Guidance on writing skills / plugin"                                                                    |
| Incremental re-translation     | 1 | "find a way to translate... new feature... without re-translating everything"                            |
| Generalize beyond Claude Code  | 1 | "Generalisation to beyond Claude Code would be great"                                                    |
| User-guided pseudocode scoping | 1 | "pseudo-code writing step should... engage with the user to see if there is functionality... not needed" |
| Maintaining non-human code     | 1 | "How can packages be updated/maintained/checked if not human-written?"                                   |
| Environment & PATH setup       | 1 | "Need to make sure to set up PATH variables correctly."                                                  |

---

## Further Considerations

- Ongoing development (do you plan on maintaining both versions?)
- Are you complying with the original license?
- Who owns the copyright?
- Preventing access to sensitive data.

## Summary

- Claude can translate code.
- Results are non deterministic.
- The plugin provides some structure, but human intervention is still required to keep Claude on track.

## With Thanks to:

James Carpenter, Tra My Pham, Asif Tamuri, David Fisher, Matteo Quartagno, Carlos Diaz Montana, Ian White, Toby Thompson

### Links:

Slides: <https://doi.org/10.5281/zenodo.22124100>

Workshop Results: <https://doi.org/10.5281/zenodo.22093626>

Stata2R Plugin: <https://github.com/stata-translations/Stata2R>